Object-Oriented Programming (OOP)

Introduction to Programming Paradigms....1
What is Object-Oriented Programming (OOP)?....2
Why was OOP Introduced?....2
Real-World Example of OOP....4
Basic Terminology....5
What is a Class?....5
Why is a Class Called a Blueprint?....6
Blueprint Example....6
What is an Object?....7
Memory Allocation....7
Class vs Object....8
Accessing Members Using the Dot (.) Operator....11
Access Specifiers....11
The Four Pillars of OOP....12
Summary....12
Practice Question....13

Introduction to Programming Paradigms

A **programming paradigm** is a style or approach to writing programs.

The major programming paradigms are:

1. Procedural Programming (C)
2. Object-Oriented Programming (C++)
3. Functional Programming
4. Generic Programming

Among these, **Object-Oriented Programming (OOP)** is one of the most widely used paradigms because it makes software easier to develop, maintain, and reuse.

What is Object-Oriented Programming (OOP)?

Definition:

Object-Oriented Programming (OOP) is a programming paradigm in which a program is designed around **objects** rather than functions.

An **object** represents a real-world entity that contains:

- **Data (Attributes/Properties)**
- **Functions (Methods/Behaviors)**

Formal Definition

> Object-Oriented Programming is a programming methodology that organizes software as a collection of interacting objects, where each object contains data and functions that operate on that data.

Why was OOP Introduced?

Earlier, most programs were written using **Procedural Programming** (like C).

As software projects became larger, several problems arose.

Problems with Procedural Programming

Imagine developing a **Hospital Management System**.

There may be:

- 500 functions
- 100 global variables
- Thousands of lines of code

Problems include:

- Difficult to manage
- Difficult to maintain
- Difficult to debug
- Data can be modified from anywhere
- Low code reusability
- Poor security

As software size increased, procedural programming became difficult to handle.

Hence, **Object-Oriented Programming** was introduced.

Why Do We Need OOP?

OOP solves many problems found in procedural programming.

Advantages

1. Better Organization

Programs are divided into classes and objects.

Example:

Student
Teacher
Employee
Library
Book

Each class manages its own data and functions.

2. Data Security

Using **Encapsulation**, data can be hidden from unauthorized access.

3. Code Reusability

Inheritance allows new classes to reuse existing code.

4. Easy Maintenance

Changes in one class usually do not affect unrelated classes.

5. Real-World Modeling

Objects closely resemble real-world entities.

6. Easy Debugging

Since programs are divided into small modules (classes), debugging becomes easier.

7. Scalability

Large applications can be expanded more easily.

Real-World Example of OOP

Consider a **Car**.

A car has:

Properties (Data)

- Brand
- Color
- Speed
- Fuel
- Engine Number

Behaviors (Functions)

- Start()
- Stop()
- Accelerate()
- Brake()
- Turn()

Thus,

Car

Data

Brand
Color
Fuel

Functions

Start()
Stop()
Brake()

The car is treated as an **object**.

Basic Terminology

There are two fundamental concepts:

- Class
- Object

Everything in OOP revolves around these two concepts.

What is a Class?

A **class** is a blueprint .

It defines:

- Data members
- Member functions

A class itself does **not** occupy memory for its data members until objects are created.

Definition

> A class is a user-defined data type that groups data members and member functions into a single unit.

Example

```
class Student
{
public:
   int rollNo;
   string name;

   void display()
   {
       cout << rollNo << " " << name;
   }
};
```

Here,

Data Members

rollNo
name

Member Function

display()

Why is a Class Called a Blueprint?

A blueprint of a house contains:

- Number of rooms
- Doors
- Windows
- Kitchen

But it is **not** an actual house.

Similarly,

A class describes:

- Variables
- Functions

but it is **not** an object.

Blueprint Example

Class

Student

Roll Number
Name
Age

display() → Object Creation

Student s1;
Student s2;
Student s3;

One blueprint can create many objects.

What is an Object?

An object is an **instance of a class**.

It represents a real entity and has its own copy of the class's data members.

Definition

> An object is a variable of a class.

Example

Student s1;
Student s2;
Student s3;

Here,

Student is the class.
s1, s2, s3 are objects.

Memory Allocation

When the class is defined:

```
class Student
{
  int roll;
  float marks;
};
```

No memory is allocated for `roll` or `marks`.

Memory is allocated only when objects are created.

Example

Student s1;

Memory is allocated for:

roll
marks

inside `s1`.

If another object is created:

Student s2;

another separate copy of `roll` and `marks` is allocated.

Class vs Object

Class	Object
Blueprint	Real entity
User-defined data type	Instance of a class
No memory for data members	Memory allocated
Defines structure	Uses the structure
One class	Many objects

Example Program

```
#include <iostream>
using namespace std;

// Define a class named Student
class Student
{
public:
    // Data members of the Student class
    int rollNo;
    string name;

    // Member function to display student details
    void display()
    {
        // Display the roll number
        cout << "Roll No : " << rollNo << endl;

        // Display the student name
        cout << "Name    : " << name << endl;
    }
};

int main()
{
    // Create an object s1 of the Student class
    Student s1;

    // Assign a roll number to the object
    s1.rollNo = 101;

    // Assign a name to the object
    s1.name = "Rahul";

    // Call the display() function using the object
```

```
    s1.display();

    return 0;
}
```

Output

Roll No : 101
Name   : Rahul
Multiple Objects

```
#include <iostream>
using namespace std;

// Define a class named Student
class Student
{
public:
    // Data member to store student's roll number
    int rollNo;

    // Data member to store student's name
    string name;
};

int main()
{
    // Create two objects of Student class
    Student s1;
    Student s2;

    // Assign values to the first student object
    s1.rollNo = 1;
    s1.name = "Amit";

    // Assign values to the second student object
    s2.rollNo = 2;
    s2.name = "Neha";

    // Display details of the first student
    cout << s1.rollNo << " " << s1.name << endl;
```

```
    // Display details of the second student
    cout << s2.rollNo << " " << s2.name << endl;

    return 0;  // Indicate successful program execution
}
```

Output
1 Amit
2 Neha

Each object has its **own independent data**.

Accessing Members Using the Dot (`.`) Operator

The **dot operator (`.`)** is used to access the data members and member functions of an object.

Example

Student s1;

s1.rollNo = 10;
s1.name = "Anita";

s1.display();

Here:

- `s1.rollNo` accesses the `rollNo` data member.
- `s1.name` accesses the `name` data member.
- `s1.display()` calls the `display()` member function.

Access Specifiers

C++ provides three access specifiers:

Access Specifier	Accessible Inside Class	Accessible Outside Class	Accessible in Derived Class
`public`	Yes	Yes	Yes
`private`	Yes	No	No (directly)
`protected`	Yes	No	Yes

By default, class members are **private**.

Example:

```
class Student
{
private:
   int rollNo;

public:
   void setRollNo(int r)
   {
       rollNo = r;
   }

   int getRollNo()
   {
       return rollNo;
   }
};
```

The Four Pillars of OOP

1. **Encapsulation** – Bundling data and functions into a single unit and restricting direct access to data.
2. **Abstraction** – Showing only the essential features while hiding implementation details.
3. **Inheritance** – Creating a new class from an existing class to reuse and extend code.
4. **Polymorphism** – Allowing the same interface (such as a function name) to behave differently depending on the object or arguments.

These concepts are typically studied after understanding classes and objects.

Summary

Concept	Description
Object-Oriented Programming (OOP)	Programming based on objects containing data and functions.
Class	A blueprint or template that defines data members and member functions.
Object	An instance of a class with its own copy of the class's data.
Data Member	A variable declared inside a class.
Member Function	A function declared inside a class that operates on the object's data.
Dot (`.`) Operator	Used to access an object's data members and member functions.
Access Specifiers	Control the visibility of class members (`public`, `private`, `protected`).
Encapsulation	Bundling data and functions together while controlling access to the data.
Abstraction	Hiding implementation details and exposing only the necessary interface.
Inheritance	Reusing and extending the functionality of an existing class.
Polymorphism	Allowing the same interface to have different implementations or behaviors.

Practice Question

Part 1: MCQs

1. What is a class in C++?

A. A variable
B. A user-defined data type

C. A function
D. An operator

Answer: B

---

2. Which keyword is used to create a class?

A. `object`
B. `struct`
C. `class`
D. `define`

Answer: C

---

3. What is an object?

A. A function inside a class
B. An instance of a class
C. A data type
D. A header file

Answer: B

---

4. Which operator is used to access members of an object?

A. `->`
B. `.`
C. `::`
D. `*`

Answer: B

---

5. Which access specifier makes class members accessible from outside the class?

A. `private`
B. `protected`

C. `public`
D. `internal`

Answer: C

---

6. What is the default access specifier of a C++ class?

A. `public`
B. `private`
C. `protected`
D. `global`

Answer: B

---

7. Which of the following correctly creates an object of class `Student`?

A. `object Student;`
B. `Student();`
C. `Student s1;`
D. `class Student s1;`

Answer: C

---

8. Consider:

```
class Student
{
public:
    int roll;
};
```

How can we assign `10` to `roll` using object `s1`?

A. `s1->roll = 10;`
B. `s1.roll = 10;`
C. `Student.roll = 10;`
D. `roll.s1 = 10;`

Answer: B

---

9. How many objects can be created from a single class?

A. Only one
B. Maximum two
C. Maximum ten
D. Any number

Answer: D

---

10. Which statement is correct?

```
class Student
{
public:
   int roll;
};

int main()
{
   Student s1;
}
```

A. `Student` is an object
B. `s1` is a class
C. `Student` is a class and `s1` is an object
D. Both are objects

Answer: C

---

11. What is stored separately for each object?

A. Class definition
B. Object's data members
C. Header files
D. Function names

Answer: B

---

12. What will be the output?

```
class Student
{
public:
   int roll;
};

int main()
{
   Student s1;
   s1.roll = 25;

   cout << s1.roll;
}
```

A. `0`
B. `25`
C. Error
D. Garbage value

Answer: B

---

13. Which operator is generally used with a pointer to an object?

A. `.`
B. `->`
C. `::`
D. `%`

Answer: B

---

14. Which of the following is a data member?

```
class Student
{
public:
   int roll;

   void display()
```

```
    {
        cout << roll;
    }
};
```

A. `Student`
B. `display()`
C. `roll`
D. `cout`

Answer: C

---

15. Which of the following is a member function?

```
class Student
{
public:
    int roll;

    void display()
    {
        cout << roll;
    }
};
```

A. `roll`
B. `display()`
C. `Student`
D. `cout`

Answer: B

Part 2: Output-Based Questions

16. What will be the output?

```
#include <iostream>
using namespace std;

class Student
{
```

```
public:
   int roll;
};

int main()
{
   Student s1;
   s1.roll = 10;

   cout << s1.roll;

   return 0;
}
```

A. `0`
B. `10`
C. Error
D. Garbage value

Answer: B

---

17. What will be the output?

```
class Student
{
public:
   int roll;
};

int main()
{
   Student s1, s2;

   s1.roll = 10;
   s2.roll = 20;

   cout << s1.roll << " " << s2.roll;
}
```

A. `10 10`
B. `20 20`
C. `10 20`
D. `20 10`

Answer: C

---

18. What will be the output?

```
class Student
{
public:
    int roll;
    string name;
};

int main()
{
    Student s1;

    s1.roll = 101;
    s1.name = "Rahul";

    cout << s1.roll << " " << s1.name;
}
```

A. `101`
B. `Rahul`
C. `101 Rahul`
D. Error

Answer: C

Part 3: Find the Error

19. Identify the error:

```
class Student
{
private:
    int roll;
};

int main()
{
    Student s1;
```

```
    s1.roll = 10;
}
```

Answer: `roll` is private, so it cannot be accessed directly from outside the class.

---

20. Identify the error:

```
class Student
{
public:
    int roll;
};

int main()
{
    Student s1;
    s1->roll = 10;
}
```

Answer: `s1` is a normal object, so use `.` instead of `->`.

Correct:

```
s1.roll = 10;
```

Part 4: Programming Questions

Basic Level

1. Student Class

Create a class `Student` with the following data members:

- `rollNo`
- `name`
- `marks`

Create one object and display all student details.

---

2. Employee Class

Create a class `Employee` with:

- `id`
- `name`
- `salary`

Create an object, assign values and display the details.

---

3. Two Student Objects

Create a `Student` class with:

rollNo
name

Create two objects `s1` and `s2`, store different student details and display them.

---

4. Rectangle Class

Create a class `Rectangle` with:

- `length`
- `breadth`

Create an object and calculate:

- Area
- Perimeter

---

5. Circle Class

Create a class `Circle` with:

- `radius`

Calculate and display the area of the circle.

Use:

Area = 3.14 × radius × radius

Intermediate Level

6. Student Result

Create a class `Student` containing:

rollNo
name
marks1
marks2
marks3

Create a member function `calculateTotal()` to calculate total marks.

Display:

Roll No
Name
Total Marks
Percentage

---

7. Employee Salary

Create a class `Employee` containing:

id
name
basicSalary

Create a member function to calculate:

HRA = 20% of basic salary
DA = 10% of basic salary
Gross Salary = Basic + HRA + DA

Display the gross salary.

---

8. Bank Account

Create a class `BankAccount` with:

accountNumber
name
balance

Create member functions:

deposit()
withdraw()
displayBalance()

Perform deposit and withdrawal operations using an object.

---

9. Number Class

Create a class `Number` containing an integer `num`.

Create member functions to check whether the number is:

- Even or odd
- Positive or negative
- Prime or not

---

10. Largest Among Two Objects

Create a class `Number` with an integer data member.

Create two objects and find which object contains the larger number.

Part 5: Slightly Challenging Problems

11. Student Grade

Create a `Student` class with:

rollNo
name
marks

Create a member function to calculate the grade:

Marks	Grade
90–100	A
75–89	B
60–74	C
40–59	D
Below 40	F

---

12. Product Class

Create a class `Product` containing:

productId
productName
price
quantity

Calculate:

Total = price × quantity

Display the complete product details.

---

13. Temperature Class

Create a class `Temperature` with a Celsius value.

Create member functions to convert:

Celsius → Fahrenheit
Celsius → Kelvin

---

14. Mobile Class

Create a class `Mobile` containing:

brand
model

---

price
ram

Create two objects and display the details of both mobiles.

---

15. Compare Two Students

Create a class `Student` containing:

name
marks

Create two student objects and compare their marks.